



MBAN EXPERIENTIAL LEARNING

BA 509 • SECTION BA2
RESEARCH PROJECT FINAL REPORT

PROJECT TITLE

AI-Agent-Based Competitor Intelligence and Customer Sentiment Analysis for Laptops

COURSE TITLE	MBAN Experiential Learning
COURSE CODE	BA 509
SECTION(S)	BA2
PROJECT TYPE	Research Project
GitHub LINK	https://github.com/anmol1722/laptop-competitor-intelligence

SUBMITTED BY

Anmol Saluja

Student ID: 21028691

1. Description of the Project and the Work Undertaken

1.1 Problem Statement

The consumer laptop market is filled with lots of products. Giant manufacturers like Dell, HP, Lenovo, ASUS, Acer, Apple, MSI, Razer, and many small companies all of them compete across sharply differentiated use-case segments: gaming machines, ultrabooks, business workstations, 2-in-1 convertibles, and budget devices. A product manager or category analyst asking an apparently simple question, "which products actually compete with ours, and what do customers say about them?" faces a genuinely hard analytical problem. Laptops differ on multiple dimensions (brand, CPU line and generation, GPU class, RAM, storage type and capacity, screen size and resolution, weight, operating system, and intended use). The competition is also complicated for example, A \$300 Chromebook and a discounted \$300 business laptop share a price point but do not compete for the same buyer.

This project built an end-to-end competitor intelligence and customer sentiment system for the laptop market. Given any laptop in the catalog, the system identifies its true competitors, benchmarks its price position within its segment and brand, mines what customers actually praise and complain about at the level of individual product attributes, and exposes all of it through an interactive web application with a natural-language question-answering agent grounded in retrieved evidence.

1.2 Tasks Undertaken

The work decomposed into seven deliverables, all of which were completed:

Component	Description	Status
Data pipeline	Cleaned, deduplicated, spec-parsed dataset of laptops and reviews	Complete
Competitor matching	Hybrid embedding + structured-spec similarity with segment/price guard	Complete
Pricing benchmarking	Price positioning within and across segments and brands	Complete
Sentiment analysis	Review-level classification and aspect-level theme extraction	Complete
Interactive application	Web dashboard for product query, comparison and insight display	Complete
RAG-based AI agent	Natural-language Q&A grounded in retrieved specs and review evidence	Complete
Final report	This document	Complete

1.3 Scope Boundary

The strategy adopted was to build a *working system across every core deliverable with analytical depth*. A substantial portion of the hallucination-mitigation goal was also delivered as a by-product of the RAG design.

2. Description of the Data

2.1 Source Selection

The proposal specified an "Amazon laptops dataset on Kaggle." Investigation of the available candidates showed this to be the wrong choice, and the decision was changed early.

The Kaggle datasets bearing this description Amazon Laptop Product Listings (July 2023), Laptop Specifications & Prices, Amazon Laptop Specs Data and similar, are small (roughly 1,000 to 4,500 rows) and, critically, contain aggregate star ratings but almost no free-text customer reviews. A dataset with a rating column and no review text column cannot support sentiment analysis, aspect-level theme extraction, or review-grounded retrieval. Three of the seven core deliverables would have been impossible.

We used the Amazon Reviews 2023 dataset released by the McAuley Lab at UC San Diego (<https://amazon-reviews-2023.github.io/>). It is an academically sourced corpus of 571.5 million reviews spanning May 1996 to September 2023, provided via HuggingFace. This choice was strictly superior on every relevant dimension: two orders of magnitude more data, genuine review text, a structured product-metadata dictionary, and a citable academic source.

2.2 Acquisition Method

The relevant category files are large, meta_Electronics.jsonl at 5.25 GB and Electronics.json at 22.6 GB, 27.9 GB in total. Rather than download and store these, a two-pass streaming filter (scripts/fetch_data.py) was written that reads each file over HTTP and writes only matching records to disk, never materialising the full corpus.

- Pass 1 streamed metadata file and retained records whose categories list contained Laptops as an exact element while excluding Laptop Accessories. Real laptops sit under Computers & Tablets > Laptops, whereas bags, skins, and decals sit under Laptop Accessories. Filtered 32,945 laptop products and their parent ASIN set.
- Pass 2 worked on the 22.6 GB review file and retained reviews for those parent_asin was in the pass 1 set. Scanning 43,886,944 reviews yielded 464,185 laptop reviews.

The streaming reader implements HTTP Range-header resumption with exponential backoff, and explicitly rejects a resumed request that returns HTTP 200 instead of 206, a silent restart would otherwise duplicate records. Total acquisition time was approximately 25 minutes for 27.9 GB, retaining 486 MB.

2.3 Data Characteristics and Constraints

After cleaning, the working dataset is **25,841 products** and **456,395 reviews** spanning 2001 to 2023.

Segment distribution:

Segment	Products
Budget	7,506
Mainstream	6,585
Business	3,953
Gaming	3,642
Ultrabook	2,434
Chromebook	1,721

Review rating distribution is strongly J-shaped, 231,217 five-star and 84,723 one-star reviews against only 36,563 three-star, with a mean of 3.73. This polarisation is analytically convenient for sentiment validation but means neutral opinion is under-represented.

The design decision was built to resolve two major data constraints:

Price is missing for 69.5% of products. Only 7875 of 25841 products have a price. The upstream metadata has no data available in this field, this might be because prices were unavailable at crawl time for delisted or out-of-stock items. The design response was a firm rule adopted throughout the system: never fabricate a

price, and never report a market statistic without its denominator. Every aggregate the system emits carries an explicit n and coverage fraction.

Brand data is not consistent. The raw store field contained case variants (acer/Acer, msi/MSI), alternative corporate names (HP vs HEWLETT PACKARD), and, most problematically, 6,843 records listing the reseller "Amazon Renewed" instead of the actual manufacturer. Treating that string as a brand would have produced a market leader larger than Dell.

3. Approach Adopted

3.1 Data Pipeline

The pipeline (src/pipeline.py) executes and performs six stages.

Accessory filtering. Despite category-level filtering, non-laptop items survive. A naive keyword filter can cause additional problems, dropping every title containing bag would discard "Panasonic Toughbook ... with Laptop Bag", a real laptop sold as a bundle. The implemented rule drops a title only when an accessory term precedes any laptop word and is not preceded by a bundle word (with, includes, +, etc.). All 122 resulting drops were manually inspected, the false-positive rate was zero. A further 46 non-laptop form factors, 42 desktops/all-in-ones and 13 oversize-screen records were removed.

Brand normalisation. Amazon Reseller were replaced by fetching the true manufacturer details from actual Brands or the product title, correcting 6,843 records. A separate boolean flag was created to represent the Renewed/refurbished status(is_renewed, 5,505 products) rather than conflated with brand. The result is 294 distinct brands with exactly one residual case collision in the entire corpus.

Specification parsing. Specifications column contains free text in a details dictionary with inconsistent coverage and formats, Processor as "2.6 GHz core_i7", RAM as "16 GB DDR4" or bare "8 GB", Hard Drive as "512 GB SSD" or bare "SSD". Rule-based parsers extract sixteen typed columns, falling back to title parsing when a details field is absent. Post-parse coverage substantially exceeds raw field coverage precisely because of this fallback:

Field	Coverage	Field	Coverage
cpu_family	93.6%	screen_in	98.3%
gpu_brand / is_discrete_gpu	100%	ram_gb	97.8%
os_family	100%	storage_gb	96.0%
cpu_ghz	87.5%	weight_lb	88.9%

Two defects found by manual review were subsequently fixed and the pipeline re-run. RAM parsing now screens candidate values against the set of capacities laptops actually ship with, and rejects a value that merely echoes the same product's drive size, a real failure mode in this corpus, where the Ram Memory Installed Size field occasionally holds the storage capacity. Weight based filter applies a screen-size-dependent floor on laptop filter, practically, a 15.6-inch screen cannot weigh 0.72 lb.

Measured effect. products with a physically impossible weight fell from 343 to 0, and disagreement between parsed ram_gb and an explicitly title-stated RAM size fell from 5.1% to 4.39%. Weight coverage fell from 91.5% to 88.9%, a deliberate trade, since a missing value is more useful downstream than a confidently wrong one.

Variant deduplication. Two variants of the same laptop with a small configuration change should be considered as one laptop instead of two. A duplication key was built from brand, a specification stripped model signature, screen size, CPU family/tier, GPU model, OS and renewed flag, deliberately excluding RAM

and storage so configuration variants collapse. This merged **6,969 rows into 3,237 model** groups, retaining the laptop variant with the most ratings. Reviews of all other variants were re-pointed to the representative of that group, preserving the original ASIN in `orig_parent_asin`.

Segment assignment was also done, Alienware, ROG, Legion, Nitro was represented into gaming. ThinkPad, Latitude, EliteBook into business and XPS, Spectre, ZenBook into ultrabook, with parsed specifications and OS.

3.2 Competitor Matching

Matching (`src/matching.py`) combines similarity signals. A product document, title, brand, segment, canonical spec phrases, price bucket is embedded with **all-MiniLM-L6-v2** into a 384-dimensional vector and cosine similarity over these is blended with a **structured-spec distance** over normalised `cpu_tier`, `ram_gb`, `storage_gb`, `screen_in`, `is_discrete_gpu` and price, at a default 60/40 weighting.

Three decision were made based on the result of the matching first draft.

Masked spec distance for missing data. With 69.5% of prices absent, a feature contributes its true distance only when observed on both products, otherwise it contributes a neutral constant at half weight. This resulted in 74.6% price-unknown against a 69.5% corpus rate, unpriced products are neither dropped nor unfairly favored.

The segment and price guard. A symmetric segment-affinity matrix sets a hard threshold along with a graded penalty, combined with a price constraint and removal of the query product and its variants.

Brand diversity cap. Initial matching code resulted in the output with either all acer or all dell laptops with no variety because title text dominates the cosine similarity. A `max_per_brand=3` cap was added.

3.3 Sentiment and Aspect Mining

Review level sentiment analysis uses `distilbert-base-uncased-finetuned-sst-2-english`, with VADER retained as fallback if the model takes a lot of time. Review representation is the mean of the score of "`<title>. <body>`" and the score of the title alone, since body-only scoring performs notably worse on 4-star reviews where a long-truncated body dilutes the verdict.

Aspect level mining defines eight different attributes, quality, thermals, value, performance, battery, display, keyboard, support. Reviews are split into buckets so that a review praising the screen while condemning fan noise yields `display=positive` and `thermals_noise=negative`, rather than one blanket polarity.

3.4 Pricing Benchmarking

The pricing module (`src/pricing.py`) computes grain and brand level medians, dispersion and percentile positions, plus a discrete-GPU premium and price-versus-rating relationship. Its descriptive characteristic is statistical validity, every aggregate carries `n`, `n_total` and coverage. Groups with fewer than five priced products are flagged `reliable: false` (this flags 261 of 294 brands), and implausible prices outside \$50–\$10,000 are identified rather than silently averaged in.

3.5 Retrieval-Augmented Generation Agent (RAG)

The RAG agent (`src/rag.py`, 1,811 lines) runs Qwen2.5-7B-Instruct locally. Retrieval reuses the embedding index rather than introducing a second indexing model. Dense retrieval fails on exact model names like "Predator Helios 300", while lexical alone fails on intent phrases like "thin and light for travel." Therefore a blend of dense similarity with TF-IDF lexical matching is used.

Context is retrieved as evidence blocks: `P_` represents product cards, `R_` verbatim review snippets and `S_` pricing statistics carrying their coverage.

The RAG prompt constraints outside knowledge, blocks arithmetic on context numbers, requires a citation per sentence, and demands an explicit "I don't know" when there is no evidence. This helps in grounding the RAG response without the hallucination. We pulled all the reviews at the end of its product resulted in much better prompt than providing separately. This prompt structure further strengthen the retrieval stopping hallucination. Prompt structure, not merely prompt wording it also determined factual accuracy. Every answer is additionally passed through a post-hoc audit that resolves each citation marker, measures sentence-level citation coverage, and verifies that every number appearing in the answer (currency, GB/TB/GHz/inch, percentages) actually occurs in the retrieved context.

3.6 Application Architecture

The interface is a FastAPI backend with a vanilla-JavaScript single-page front end. It comes with laptop search and filtering, competitor comparison analysis tab with a guard on/off ablation toggle, pricing positioning annotated with coverage, sentiment analysis for product divided into category, and a chat panel for the RAG agent with clickable citations.

Laptop Competitor Intelligence
Matching · price positioning · grounded review evidence

25,841 LAPTOPS
distinct listings in products.parquet

294 BRANDS
across 6 segments

456,395 REVIEWS ANALYSED
of 456,395 retained (100.0%)

30.5% PRICE COVERAGE
7,875 of 25,841 listings priced

100.0% SENTIMENT COVERAGE
25,831 of 25,841 products mined

API ok · caches warm

Find a product Reset

Search text
e.g. thinkpad t14, rtx 4060, macbook air

Brand: any Segment: any

Min price (\$): — Max price (\$): —

Min RAM: any Discrete GPU: any

☐ Only products with mined review sentiment (26.5% of catalogue)

☐ Exclude renewed / refurbished listings

Sort: Relevance

Search

25,841 of 25,841 listings match 30% priced - 66 ms

• No query text, so 'relevance' falls back to most-reviewed first.

Acer Aspire 3 A314-23P-R3QA Slim Laptop | 14.0" Full HD IPS Display | AMD Ryzen 5 7520U Quad-Core Processor | AMD Radeon Graphics...
Acer · mainstream · B0C2ZMJWS3

Overview Competitors Pricing Reviews

Specification B0C2ZMJWS3

CPU	AMD Ryzen 5 5	CPU CLOCK	4.3 GHz
RAM	8 GB LPDDR5	STORAGE	512 GB SSD
SCREEN	14.0" · 1920x1080	GPU	Radeon Graphics · integrated
OS	Windows	WEIGHT	3.1 lb
STORE	Acer	VARIANTS	1

Fields shown as *not parsed* were absent or unparseable in the source listing — they are unknown, not zero.

Price position vs its segment — mainstream \$439.99

this product \$439.99

Review sentiment mined

OVERALL POLARITY	0.30 (positive)	POSITIVE SHARE	68%
MEAN REVIEW RATING	3.97	REVIEWS SCORED	5,008 of 5,008 retained

Only 100.0% (25,831 of 25,841) of the catalogue has mined sentiment: a full pass is queued.

Ask the analyst model idle

Answers are generated only from retrieved catalogue evidence, then audited.

Every answer ships with its evidence. Citation markers such as [P1] (product), [R1] (review) and [S1] (market statistic) are clickable — they open the exact record the model was shown.

An audit runs on each answer: invented citation markers, numbers that appear in no evidence block, and reviews attributed to the wrong product are all reported.

Best gaming laptop under \$1200?

Lenovo vs HP on price

Top complaints in budget laptops

The 7B model is not resident yet: your first question also loads it (~30-60 s), after that answers take about 2-17 s.

Ask about products, prices, competitors or what reviewers say...

Enter to send · Shift+Enter for a new line **Ask**

4. Delivery Outcomes

4.1 Competitor Matching

Rigorous testing was performed for example the system should not pair a low-budget Chromebook with a discounted business laptop. This was tested by ablation rather than asserted. Across 200 query products × top-10 results (2,000 recommendations):

Configuration	Cross-class matches	Matches beyond 3× price gap
Guard disabled	36	15
Guard enabled	0	0

This sweep was re-executed after the parser fixes of error mentioned in section 5.3 and the re-embedding, the conclusion is unchanged, with counts shifting slightly from 38/17 to 36/15 because corrected ram_gb and storage_gb values alter the structured-spec distances. An independent adversarial check across a further 500 queries (5,000 returned competitors) found 100% segment purity and could not construct a single Chromebook-into-premium leak.

The qualitative behaviour is equally clear. For a \$300 Panasonic Toughbook (business), the unguarded system returned an Acer Chromebook 14 (\$298), an Acer Chromebook 314 (\$180) and a Cloudbook 14 (\$239) with price-plausible, competitively meaningless. With the guard enabled it returned the Toughbook CF-30 and CF-31, a Dell Latitude 7390, an HP EliteBook 840 G5 and a ThinkPad T460S.

4.2 Sentiment Classification

Validated against star ratings as ground truth (1 to 2 negative, 4 to 5 positive, 3 excluded) on a **stratified sample of 6,000 reviews, 1,500 per star**, with a fixed seed and **no threshold tuning**:

Backend	Accuracy	Macro F1	Precision (neg)	Recall (neg)
DistilBERT SST-2	0.890	0.890	0.862	0.930
VADER	0.791	0.788	0.876	0.677

Restricting to confident predictions polarity > 0.2 raises accuracy to **0.926 at 83.5% coverage**. Review-level polarity correlates 0.794 with star rating; product-level positive share correlates 0.77 with mean rating.

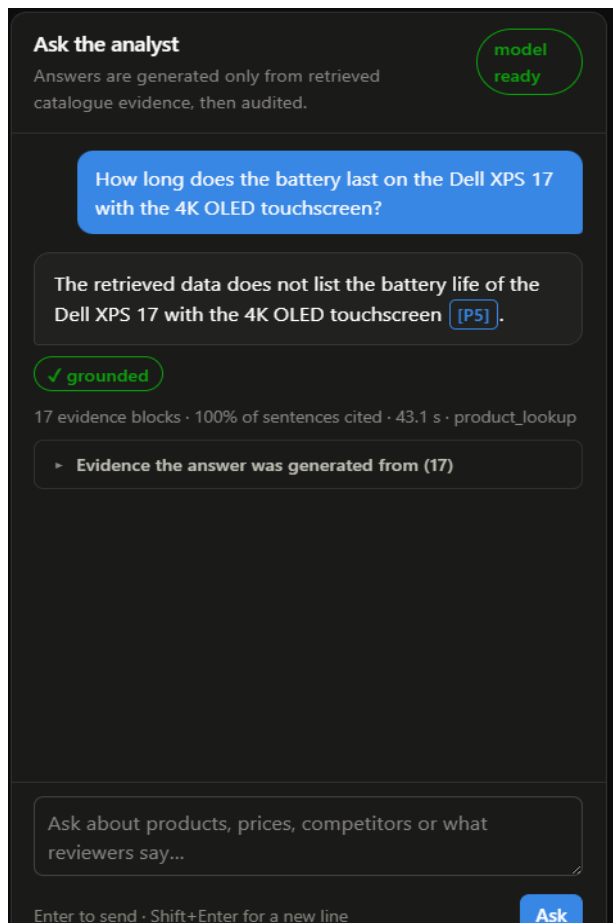
The classifier was subsequently run over the **entire 456395 review corpus** on GPU, yielding aspect-level sentiment for **25831 of 25841 products (99.96%)**. The ten products without sentiment have no retained review text.

An independent adversarial reviewer re-executed this validation end to end and reproduced the reported figures to every digit (0.89), confirming no label leakage.

4.3 RAG Agent

We tested five representative question types, citation coverage was well over 75%, with no spurious citation and no cross-attributions in the manual test. Refusal behaviour on out-of-corpus questions works.

Asked "How long does the battery last on the Dell XPS 17 with the 4K OLED touchscreen?", battery life being a field never collected, the agent answered: "The retrieved data does not include the battery life of the Dell XPS 17..." rather than confabulating a plausible number. The same held for a benchmark-score question.



Critically, the audit layer was correctly firing on genuine model errors, not on tests. During end-to-end verification the model invented markers [R13], [R14] and [P7], all of which were correctly captured as unsupported_markers and surfaced in the interface as warning-flagged citations. This constitutes substantial delivery of what the proposal listed as goal.

4.4 Pricing Benchmarking

All five validation checks pass. Segment price ordering is monotonic and economically possible:

Segment	Median	Priced n	Coverage	p10-p90
Chromebook	\$209.00	769	44.7%	\$81-\$401
Budget	\$319.95	1,401	18.7%	\$184-\$499
Business	\$339.99	1,948	49.3%	\$159-\$1,199
Mainstream	\$649.99	1,926	29.2%	\$463-\$1,000
Ultrabook	\$799.00	852	35.0%	\$360-\$1,354
Gaming	\$1,369.99	979	26.9%	\$800-\$2,597

4.5 Substantive Market Findings

We were able to produced genuine market insight:

A discrete GPU commands a \$751 median premium (+167%) \$1199.99 median for discrete-GPU laptops versus \$449.00 for integrated, a significant difference at $p < 0.001$ (Mann-Whitney). This is the single largest price determinant identified.

Price is nearly uncorrelated with customer satisfaction. $r = 0.118$ (Spearman $\rho = 0.170$) between price and average rating across 6410 priced products. Spending more does not reliably buy a better-reviewed machine.

Aspect-level sentiment reveals a consistent industry-wide pattern. Aggregating mention-weighted positive share across the full 456395-review corpus (25831 products):

Aspect	Mentions	Products	Positive share
Build quality	36,282	8,985	61.7%
Value / price	109,012	15,113	61.5%
Performance	137,102	17,382	58.3%
Display	107,512	15,233	51.6%
Keyboard / trackpad	77,454	12,971	45.5%
Thermals / noise	32,332	8,373	40.6%
Battery	74,244	12,577	37.0%
Support / service	54,225	11,985	17.7%

The conclusions proved stable. The stratified sample was then compared with all the data corpus to validate the sample. Every aspect share stayed within 1.7 percentage points and the ranking was preserved. This is useful evidence that stratified sampling was an adequate basis for the interim conclusions, even though the final figures rest on the complete corpus.

The two weakest attributes are not hardware at all in one case: **customer support is overwhelmingly negatively discussed (17.2% positive)**, and **battery life is the weakest hardware attribute (36.2%)**. For a manufacturer, this is directly actionable, the marginal return on improving after-sales service appears far higher than on further raw performance, which is already the most-discussed and third best-received attribute.

Brand price positioning. MSI with \$1300 median and ASUS (\$700) is in the performance tier. HP (\$499), Lenovo (\$529) and Dell (\$465) are in the middle by pricing. Gateway (\$224) are the low cost laptops. Mean ratings across major brands span a narrow band (3.58 to 3.95), reinforcing that price does not correlate with satisfaction.

5. Technical and Non-Technical Challenges

5.1 Require Nvidia GPU.

The generated-chat feature uses the **4-bit Qwen2.5-7B-Instruct** model. Although quantization reduces memory usage, the model still requires approximately 6 GB of GPU memory and is designed to run with **NVIDIA CUDA**.

Before loading the model, the API checks GPU memory using `nvidia-smi`. On Apple laptops with no GPU, this command is unavailable because the machine does not contain an NVIDIA GPU. **Therefore, a Nvidia GPU is required to run this project.**

5.2 The Streamlit Failure and Architecture Change

The proposal specified Streamlit, and a Streamlit application was in fact built (`src/app.py`, 1,563 lines). It could not be made reliable. The root cause is an incompatibility between **pandas 3.0 / PyArrow's ``mimalloc`` allocator and Streamlit's script-rerun threading model**: PyArrow's thread-local heaps are created and destroyed as Streamlit's script-runner threads cycle, producing segmentation faults when Arrow-backed columns are touched across reruns.

Rather than continue debugging a framework-internal defect, the interface was rebuilt with the help of Codex as a **FastAPI backend plus a vanilla-JS front end**.

5.3 Defects Found by Adversarial Verification

A deliberate methodological choice was to subject each completed module to **adversarial review that re-executed the code against real data**. This proved highly valuable and surfaced defects that unit tests and visual inspection missed.

Two genuine defects were confirmed by independent reproduction, and both have since been fixed.

RAM/storage column contamination. The Ram Memory Installed Size field occasionally holds the *drive* capacity rather than the memory size. Cross-checking against titles that state RAM explicitly showed **646 of 12,655 such products (5.1%) had a parsed `ram_gb` disagreeing with their own title**, one product titled "...4GB RAM..." was parsed as 128 GB, which was in fact its storage size. Because matching.py uses ram_gb in its spec distance, this propagated into competitor rankings.

The fix is worth describing, because **the diagnosis offered by the reviewer was wrong and following it would have made the system worse**. The reviewer proposed reading the RAM field before Ram Memory Installed Size. Measured against title-stated RAM as ground truth across 17,405 products, that reordering *reduced* agreement from 95.08% to 93.00%: where the two fields disagree, Ram Memory Installed Size is actually correct 66.3% of the time against RAM's 30.2%. The defect was narrower than diagnosed. The implemented fix therefore preserves the field order and instead screens candidate values, rejecting capacities no laptop ships with, and rejecting any value that merely echoes the same product's drive size (threshold tuned empirically over 16/32/64 GB). Title disagreement fell to 4.39% and impossible values (≥ 128 GB) from 42 to 24.

Taken together, these episodes support a single methodological conclusion: **an automated reviewer's findings are hypotheses, not results**. One finding was real but misdiagnosed, one was real and correctly diagnosed, and one was entirely spurious despite a confident CRITICAL severity rating. Accepting any of them at face value would have caused hallucinations and incorrect answers. In each case the deciding factor was measuring the proposed change against an independent ground truth before adopting it.

5.4 Analytical Challenges

Balancing multiple similarity dimensions. No single feature defines laptop competition. The hybrid text-plus-spec approach with segment/price guarding was the resolution, and the max_per_brand cap, was necessary to make results useful.

Missing data without distortion. With 69.5% of prices absent, the choice was between imputation (risking fabricated market statistics) and exclusion (discarding two-thirds of the catalogue). The adopted compromise: prices are never imputed for reporting or for the similarity score, but a hierarchical median estimate is used *solely* for the price-band guard and for display, always flagged as estimated.

6. Personal Reflection on MBAN Program Learnings

Four areas of MBAN coursework were directly applied.

BAIT_V 509 Business Applications of Machine Learning. The competitor-matching module is fundamentally a representation-learning and ranking problem. Coursework on similarity metrics, embedding spaces and the treatment of mixed categorical/continuous features translated directly into the hybrid scoring design. The most transferable lesson was one about evaluation rather than modelling, the ablation comparing guard-on

against guard-off is what turned "the matching seems reasonable" into a defensible quantitative claim (38 cross-class matches versus 0). Building the counterfactual comparison mattered more than tuning the model. **BAMA_V 520 Topics in Marketing and BAIT_V 508 Business Analytics Programming.** Sentiment classification, clause segmentation and aspect-level opinion mining were applied throughout. The key practical insight was that document-level sentiment is the wrong unit of analysis for product reviews, a laptop review routinely praises one attribute and condemns another, and collapsing that into a single polarity destroys precisely the information a manufacturer needs. Aspect-level extraction despite its imperfections is what made the finding that support/service sits at 17.2% positive while build quality sits at 62.5% possible at all.

BAMA_V520 Customer analytics. Segmentation, price positioning and competitive benchmarking framed the entire project. The discipline of always reporting a denominator came from statistical training, and it materially changed the output: with 69.5% of prices missing, an unlabelled "average laptop price" would have been actively misleading. The finding that price and satisfaction are nearly uncorrelated ($r = 0.118$) is a marketing-analytics result, not a technical one, and it is the kind of counter-intuitive conclusion that only emerges from disciplined measurement.

BAMS_V 521 Analytics Leadership. Relating specification and experience factors to customer response the discrete-GPU premium, aspect-level satisfaction patterns, brand price stratification against a narrow satisfaction band is diagnostic analytics applied to a real, messy commercial dataset.

Data quality is at most importance for a stable robust model. The most consequential engineering decisions were not modelling choices but data decisions, recognising that 6843 records attributed to a reseller would corrupt brand analysis, and that RAM and storage fields could be transposed. The RAM parsing defect is instructive precisely because it is unglamorous and would silently degrade every downstream result. Verification must be adversarial and independent. Modules that passed their own tests failed under manual review that re-executed them against real data and one review finding was itself wrong. Both directions of that lesson matter. Honest reporting of limitations strengthens rather than weakens analytical work. Explicitly flagging 261 of 294 brands as statistically unreliable makes the remaining 33 trustworthy.

7. Conclusion and Lessons for the Future

7.1 Conclusion

A complete, functioning competitor-intelligence system covering all seven core deliverables was built from scratch. It ingests 27.9 GB of raw Amazon data, reduces it to 25841 cleaned and spec-parsed laptops with 456395 reviews, identifies competitors with demonstrably zero cross-segment leakage, benchmarks prices with explicit statistical honesty, mines review sentiment across all 456395 reviews at 89.0% validated accuracy, and answers natural-language questions through a locally-hosted 7B language model that cites its evidence and refuses to answer when evidence is absent.

The important market findings, a \$751 discrete-GPU premium, near-zero price/satisfaction correlation, and customer support as the industry's weakest attribute at 17.7% positive sentiment across 54,225 mentions, demonstrate that the system produces genuine competitive insight.

7.2 Limitations

1. **Competitor matching lacks ground truth.** A labelled set of 40 laptop pairs evaluated by P@k and nDCG needs to be build. Validation instead rests on the guard ablation and segment-purity

measurement, which demonstrate a specific failure mode is eliminated but do not measure ranking quality against human judgement.

2. **RAG evaluation is informal.** A 20-question test set scored 1 to 5 for faithfulness and relevance can be constructed, currently evidence rests on a five-question self-test plus adversarial probing. Additional Human in the loop testing is required to improve the prompt quality and build better RAG. The agent's grounding behaviour is supported by its builder's own instrumentation plus incidental observation during interface testing, not by an independent adversarial pass.

7.3 Lessons and Future Work

The idea of filtering the data before downloading made the code lean and saved a lot of data cleaning overhead.

Measure the environment before committing to it. The Ollama decision was reversed on a single measurement, 15 MB/s versus 0.3 MB/s. Ten seconds of measurement saved significant run time.

Prompt structure is a first-class variable. Interleaving review evidence with its parent product eliminated citation cross-attribution that no amount of instruction rewording had fixed.

Build the ablation, not just the feature. The guard's value is established entirely by the 38-versus-0 comparison.

Future work, in priority order:

1. The 40 to 60 pair labelled ground truth and report Precision@K (P@K) and Normalized Discounted Cumulative Gain (nDCG) for competitor ranking needs to be constructed and validated.
2. Build the 20-question RAG evaluation set and re-run the independent grounding verification
3. Add stemming and polysemy disambiguation to aspect cue matching and widen clause splitting beyond the current conjunction set.
4. The natural extensions are the descoped stretch goals a formal confidence-scoring framework and cross-segment competitive dynamics analysis, followed by temporal analysis exploiting the 2001 to 2023 review span to track how competitive positioning and customer complaints evolve.
5. Update the RAG with the data from 2023 onwards.

Appendix A: System Artefacts

Component	File
Shared schema contract	src/config.py
Data acquisition	scripts/fetch_data.py
Data pipeline	src/pipeline.py
Competitor matching	src/matching.py
Pricing benchmarking	src/pricing.py
Sentiment & aspects	src/sentiment.py
RAG agent	src/rag.py
REST API	src/web/api.py
API test suite	src/web/test_api.py
Front end	src/web/static/

GitHub Link for code: <https://github.com/anmol1722/laptop-competitor-intelligence>

Data artefacts: products.parquet (25,841 × 28), reviews.parquet (456,395), product_embeddings.npy (25,841 × 384), product_sentiment.parquet, review_sentiment.parquet.

Evaluation artefacts: eval/sentiment_eval.json, eval/pricing_eval.json.

To run: cd project && python -m src.web.api

then open <http://127.0.0.1:8000/>.

Appendix B: Data Source Citation

Hou, Y., Li, J., He, Z., Yan, A., Chen, X., & McAuley, J. (2024). *Bridging Language and Items for Retrieval and Recommendation*. Amazon Reviews 2023 dataset, McAuley Lab, UC San Diego. <https://amazon-reviews-2023.github.io/>